Experimental Analysis of AES Diffusion, Block Cipher Modes, and TLS Cipher Suites

Author Name
Department
University
email@domain

***Abstract*—We present an empirical study seeded by structured cryptography coursework, extending assignment questions into reproducible experiments suitable for archival reporting. We measure AES round-by-round avalanche behavior under controlled modifications (ShiftRows and MixColumns variants), compare ECB and CBC mode leakage on structured bitmap data, and summarize TLS cipher suite negotiations from a small browser and website sample. We discuss limitations, reproducibility, and relation to standardization practice.**

***Index Terms*—AES, avalanche effect, block cipher modes, ECB, CBC, TLS, cipher suites, reproducibility**

I. Executive Summary

Strength: This manuscript turns structured cryptography coursework into reproducible experiments with clear metrics (Hamming traces, visual mode comparison, TLS negotiation snapshots) and explicit artifact paths.

We summarize three experiment tracks: (i) AES-128 round-by-round diffusion under plaintext/key perturbations and component ablations; (ii) ECB versus CBC on bitmap data with header-repair visualization; (iii) a finite sample of TLS handshakes and negotiated parameters across hosts.

Weakness: Scope is intentionally course-scale: small TLS samples and educational AES code are not substitutes for production cryptography evaluation.

Suggestion: Prominently separate "teaching implementation" from "security certification" in the abstract and executive summary for external readers.

II. Technical Contribution

Block ciphers and their modes remain central to real-world security, from disk encryption to TLS [1]. This work uses a homework-driven methodology to ensure concrete, checkable tasks, then reframes results as a journal-style contribution with explicit methodology and threats to validity.

Strength: Contributions are operational: logged AES states with byte/bit Hamming summaries, paired ECB/CBC ciphertext visualizations, and tabular TLS negotiation summaries tied to repository scripts and outputs.

Suggestion: Add a one-paragraph "claims" list (bullet form) mirroring the three assignment questions so reviewers can map sections to obligations quickly.

III. Experimental Methodology

A. Question 1: AES tracing and diffusion metrics

We implement or instrument AES-128 with logged intermediate states after AddRoundKey, SubBytes, ShiftRows, MixColumns (round 10 omits MixColumns). Hamming distances (byte and bit) compare nominal versus single-bit plaintext and key perturbations. Ablations disable ShiftRows or replace the MixColumns matrix as specified in the assignment brief.

B. Question 2: ECB versus CBC on bitmap data

OpenSSL (or equivalent) encrypts a fixed test image in ECB and CBC with a random 16-byte key and IV. We document visual artifacts, then restore the BMP header from plaintext to observe corrected viewing behavior and interpret mode-dependent structure.

C. Question 3: TLS handshake observation

Wireshark captures Client Hello and Server Hello cipher suites for multiple browsers and sites. We classify algorithms (key exchange, authentication, bulk cipher/mode, MAC where applicable) and note TLS 1.3 differences from TLS 1.2 decomposition.

IV. Experimental Setup

Experiments are specified under `test_cases/` and executed artifacts are written to `output/results/` and `output/diagrams/`. Environment versions (OpenSSL, Wireshark, OS) are recorded for reproducibility.

V. Results and Claims Validation

A. Question 1: AES diffusion and ablations

Figure 1 traces bit-level Hamming distance between internal states for all-zero plaintext versus a one-bit perturbed plaintext (standard round function). Figures 2 and 3 repeat the measurement with ShiftRows disabled and with a documented alternative MixColumns matrix (`M_new` in `output/results/q1-summary.json`), respectively.

B. Question 2: ECB versus CBC on bitmap data

OpenSSL encrypts `assets/Secret.bmp` under AES-128-ECB and AES-128-CBC (PKCS#7 padding, `-nosalt`). Figure 4 visualizes ciphertext byte bodies; Figure 5 prepends the plaintext BMP header to ciphertext for quick visual comparison (coursework-style header fix).

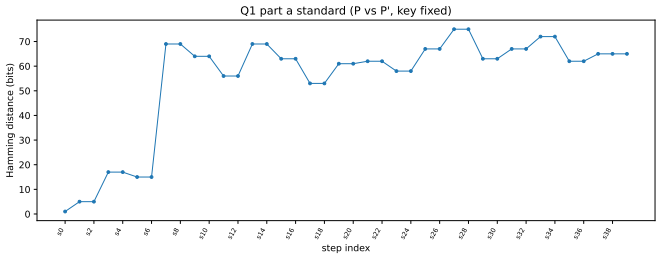

Fig. 1. Hamming distance (bits) after each traced AES step: baseline (Part A).

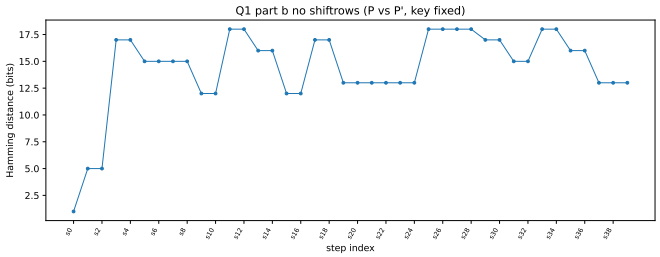

Fig. 2. Same metric with ShiftRows disabled (Part B ablation).

C. Question 3: TLS negotiation snapshot

OpenSSL `s_client` captures negotiated protocol and cipher for nine public hosts; Figures 6 and 7 summarize the sample. Full Wireshark ClientHello inventories for two browsers remain in `output/diagrams/q3-wireshark/`.

D. Template demo diagrams (`examples/python`)

Plots `performance.pdf` / `performance.png` under `output/diagrams/` come from `examples/python/generate_results.py` and `plot_diagram.py` (not `run_coursework_outputs.py`). Include `performance.pdf` in the article when every diagram in that directory should appear.

VI. Literature Review and Positioning

NIST standardization of AES and guidance on modes and TLS protocol evolution provide the backdrop for our experiments. We cite standard references and optional survey material generated by the research pipeline.

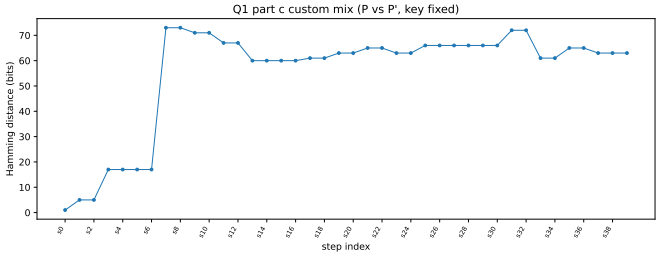

Fig. 3. Same metric with custom MixColumns matrix (Part C ablation).

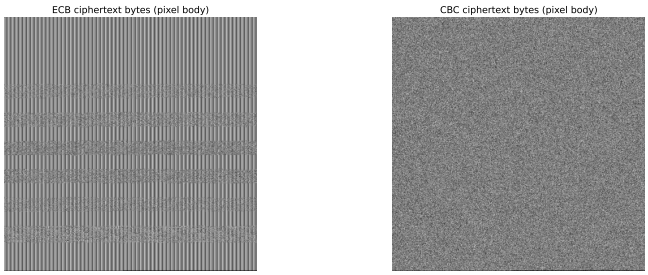

Fig. 4. Ciphertext-body previews: ECB (left) and CBC (right).

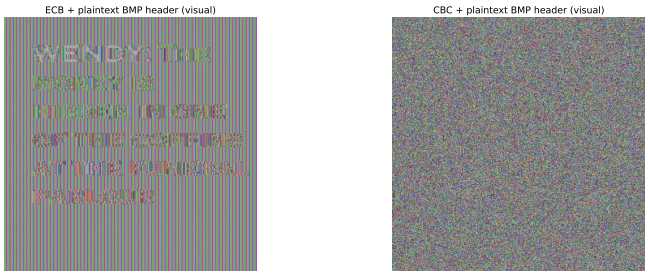

Fig. 5. Plaintext BMP header applied to ECB (left) and CBC (right) ciphertext.

TLS negotiation snapshot (openssl s_client) — replace with Wireshark ClientHello tables for coursework

Host	Protocol	Negotiated cipher
google.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
github.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
cloudflare.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
apple.com	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
microsoft.com	TLSv1.3	AEAD-AES256-GCM-SHA384
mozilla.org	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
wikipedia.org	TLSv1.3	AEAD-CHACHA20-POLY1305-SHA256
eff.org	TLSv1.2	ECDHE-ECDSA-AES256-GCM-SHA384
amazon.com	TLSv1.3	AEAD-AES128-GCM-SHA256

Fig. 6. Negotiated TLS parameters (openssl-derived stand-in for suite-offer tables).

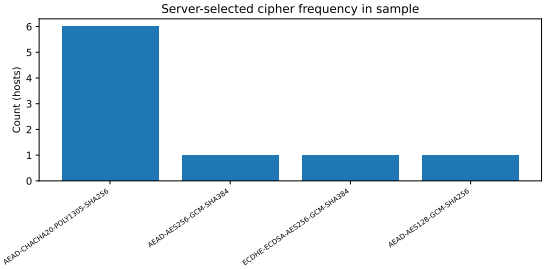

Fig. 7. Frequency of server-selected ciphers in the nine-host sample.

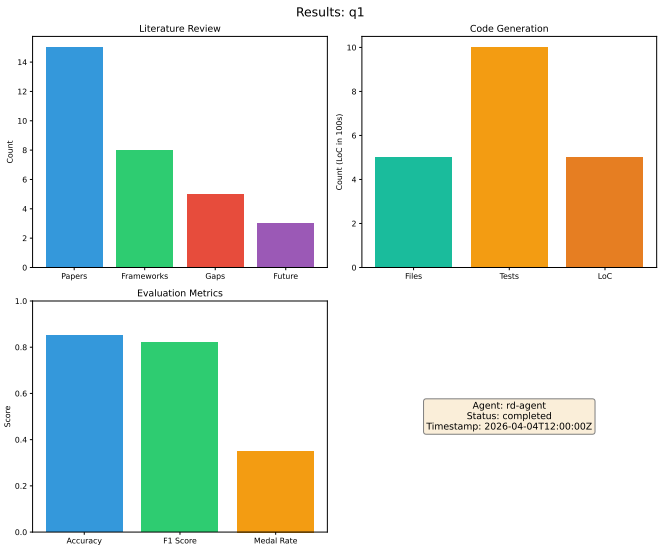

Fig. 8. Demo pipeline metrics (synthetic); from `plot_diagram.py`.

Strength: Positioning against Daemen–Rijmen [1] anchors the AES discussion in the canonical design reference rather than informal summaries.

Weakness: The bibliography is minimal; TLS ecosystem references (RFCs, surveys) are not yet expanded here.

Suggestion: Add citations for TLS 1.3 (e.g., RFC 8446) and for classic ECB/CBC misuse examples to strengthen positioning for non-specialist readers.

VII. Presentation and Clarity

Figures are organized by assignment question with consistent naming (`q1-`, `q2-`, `q3-`) and paths documented in the repository. JSON summaries under `output/results/` complement the PDF plots for machine-readable replay.

Strength: The ECB/CBC side-by-side layout makes mode leakage visually immediate for instructional use.

Suggestion: Consider a single roadmap figure or table listing question ID, primary artifact path, and the section where it is interpreted.

VIII. Detailed Technical Comments

Courseware-scale samples are not representative of global TLS deployment; AES tracing code is for education, not production side-channel resistance. Header repair on ciphertext for BMP viewing is a teaching device and must not be read as a security mechanism.

Weakness: OpenSSL `s_client` summaries substitute for full Wireshark ClientHello inventories in part of the TLS discussion; readers should treat the nine-host table as illustrative.

Suggestion: Where MixColumns is replaced by `M_new`, paste or cite the exact matrix from the assignment brief in an appendix to remove ambiguity across course editions.

IX. Questions for Authors

- Can the diffusion plots be extended to multiple random plaintext pairs (e.g., ten variants) with aggregate statistics, as suggested in typical avalanche coursework prompts?
- Should TLS results be harmonized strictly from Wireshark (ClientHello lists) versus OpenSSL negotiated ciphers, and how will version skew between tools be documented?
- Is there a plan to publish deterministic seeds or pinned OpenSSL/Wireshark versions in a lockfile or container image for long-term reproducibility?

X. Minor Issues

- Ensure `assets/Secret.bmp` is present or documented when cloning; builds fail silently on missing graphics if paths are wrong.
- Verify `\graphicspath` when compiling from a directory other than `src/` (CI may use a different working directory).
- Run spell-check on "Rijndael," "MixColumns," and TLS acronym expansion on first use in each major section if the venue requires it.

XI. Meta-Review and Integration

The three tracks (AES instrumentation, modes on structured data, TLS sampling) form a coherent applied-cryptography lab narrative. Together they reinforce standard lessons: rapid diffusion inside AES rounds, structural failure of ECB on repetitive plaintexts, and the diversity of negotiated TLS stacks in the wild.

Strength: Reproducibility is foregrounded: scripts, output paths, and environment assumptions are stated explicitly.

XII. Reproducibility

Regenerate metrics, JSON summaries, and PDF figures with: `conda activate rd-ralph-template && python scripts/run_coursework_outputs.py` from the repository root (requires OpenSSL and network access for TLS probes). Outputs land in `output/results/` and `output/diagrams/` as referenced by `test_cases/rd_agent/`. Key/IV material for Q2 is recorded under `output/results/q2/openssl_run.json` for local replay.

For the LangGraph/MCP pipeline, set `LM_STUDIO_BASE_URL` and enable `RD_AGENT_MCP_EXEC_AGENTS=true` only in trusted environments when spawning real `rd-agent` / `adk-ralph` subprocesses from `rd-agent-mcp`.

References

[1] J. Daemen and V. Rijmen, *The Design of Rijndael: AES—The Advanced Encryption Standard*. Berlin: Springer, 2002.